Project 4: Classification with k-Nearest Neighbors

Part I: Heart Disease Prediction | Part II: Dry Bean Classification

This report describes how we tested k-nearest neighbors on two different datasets. In both parts, we focused on choosing a useful set of features and a value of k, then checking how well the resulting models made predictions on held-out data.

Presentation:
https://docs.google.com/presentation/d/1TpZa9XU2TqRL4rr5XwWAoTOX4bvYj2Sg0kWZArMxeCw/edit?usp=sharing

Git Repo:

https://github.com/sjhardy1/project4

Part I — Heart Disease Prediction

Introduction

For this project, we wanted to see whether we could use patient information to predict if someone has heart disease. We worked with the Cleveland Heart Disease dataset from the UCI Machine Learning Repository. It originally has 303 patients and 13 features, such as age, chest pain type, cholesterol, maximum heart rate, and results from medical tests. The diagnosis is recorded from 0 to 4. We changed it into two groups: 0 means no heart disease, and 1–4 means heart disease.

We chose k-nearest neighbors (kNN) because the basic idea is easy to follow: find the patients who are most similar to a new patient, then use their diagnoses to make a prediction. The main challenge was deciding which patient features mattered most and how many neighbors (k) to use.

Methods

Preparing the data

Six rows had missing values in the ca or thal columns, so we removed them. That left 297 patients. We used sklearn's train_test_split to divide them into 237 training patients (80%) and 60 test patients (20%). We kept the test set separate while choosing the model settings so we could check performance on patients the model had not used for tuning.

Before measuring distances, we also prepared the features. We one-hot encoded categorical features (cp, restecg, slope, and thal), kept the binary features (sex, fbs, and exang) as 0 or 1,

and standardized numerical features such as age, cholesterol, and blood pressure. This matters because kNN uses distances: without scaling, a feature with large numbers could outweigh everything else. We calculated the scaling values using only the training data.

Building and choosing the model

We built our own kNN classifier using sklearn's NearestNeighbors rather than KNeighborsClassifier. For each patient, it finds the k nearest training examples and predicts the majority class. We tried odd k values from 1 to 41 to avoid ties.

First, we used all 13 features as a baseline. Then we used forward selection, adding one feature at a time and keeping the one that improved the cross-validated F1 score the most. At each step, GridSearchCV compared k values using 10-fold cross-validation on the training set. This let us compare small feature sets with larger ones instead of assuming all 13 features were helpful.

F1 score was our main measure because it combines precision (how often a disease prediction is correct) and recall (how many disease cases the model identifies).

We used the training data to compare model choices rather than picking settings based on how well they worked on the final test patients. For forward selection, we checked which unused feature improved the average validation score most at each step. We recorded the best value of k for each feature-set size instead of assuming one value would work for every combination.

Results

The baseline model using all 13 features reached a cross-validated F1 score of 0.800, with k = 7. Forward selection performed better. Its highest score was 0.8539 with seven features, and the best k at that step was 41.

The selected features were thal, ca, cp, slope, exang, fbs, and trestbps. Interestingly, these include chest pain and several test or exercise-related measurements. The score improved a lot while adding the first few features, then generally dropped as more were included. That suggests extra features were not always useful for measuring which patients were similar.

This result is useful because kNN gives every included measurement a role in the distance calculation. A feature can seem informative on its own but still make the neighbor search worse if it adds a lot of variation that is not helpful for predicting the outcome. Our comparison therefore looked at both the number of features and the prediction score, rather than simply keeping all available columns.

Model	Best k	CV F1 (training)
All 13 features	7	0.8000
Selected 7 features	41	0.8539

Table 1. Baseline compared with the selected feature set.

Test-set performance

After choosing the seven features and k = 41, we used 10-fold cross-validation to train 10 models, each on a different nine-tenths of the training set. We then evaluated each model on the same 60 test patients. Table 2 shows every score, along with the averages.

Fold	Precision	Recall	F1
1	0.7586	0.9565	0.8462
2	0.7857	0.9565	0.8627
3	0.8148	0.9565	0.8800
4	0.7419	1.0000	0.8519
5	0.7778	0.9130	0.8400
6	0.8077	0.9130	0.8571
7	0.8077	0.9130	0.8571
8	0.8462	0.9565	0.8980
9	0.7857	0.9565	0.8627
10	0.8148	0.9565	0.8800
Mean	**0.7941**	**0.9478**	**0.8636**

Table 2. Precision, recall, and F1 on the test set for all 10 fold models.

Recall was especially high: depending on the fold, the model identified 21 to 23 of the 23 patients with disease in the test set. Precision was lower, meaning it also incorrectly marked some healthy patients as having disease. In a screening situation, catching more possible cases can be helpful, although false alarms still matter and the model is not a medical diagnosis tool.

Discussion of results

One thing that stood out was that adding more features did not always improve kNN. Our selected seven-feature model had a higher training cross-validation F1 score than the full-feature baseline (0.854 versus 0.800). It also gave a mean test-set F1 of about 0.864, which is encouraging for this dataset.

There are limits to what we can conclude. The dataset is small, and the same 60 test patients were evaluated by all 10 fold models, so those test results are not ten independent evaluations. Also, k = 41 was the largest value we tried; checking higher values could be useful. Overall, this project helped us understand how scaling, feature selection, and choosing k affect a distance-based classifier.

For the class challenge, the assignment also provides a small sample test file. The intended next check is to train the chosen approach on the labeled Cleveland data, make predictions for each sample patient without cross-validation on the sample, and calculate F1. We do not report a sample challenge score here because it has not been provided.

Part II — Dry Bean Classification

Introduction

For Part 2, we wanted to see how the same kNN approach would work on a different dataset. We used the Dry Bean dataset from the UCI Machine Learning Repository. It contains 13,611 beans belonging to seven types: Dermason, Sira, Seker, Horoz, Cali, Barbunya, and Bombay. Each bean was photographed, and 16 measurements of its size and shape were recorded, including area, perimeter, roundness, and shape factors. Unlike Part 1, which had two possible outcomes, this task requires choosing among seven classes.

Dataset

There were no missing values, and all 16 features were numerical, so we did not need to remove rows or use one-hot encoding. We used sklearn's train_test_split to divide the data into 10,888 training beans (80%) and 2,723 test beans (20%). We used a stratified split so each bean type had roughly the same proportion in both sets. This was useful because Bombay beans made up only about 4% of the dataset.

We standardized the measurements using statistics calculated from the training data only, then applied the same transformation to the test data. This matters for kNN because some measurements, such as area, are much larger than the shape factors and could otherwise dominate the distance calculation.

Keeping the test examples separate was important here as well. If the mean and standard deviation had been calculated using the full dataset, information from test cases would have influenced the way training examples were represented. We avoided that by fitting the scaling step on the training portion.

Methods

We reused our own kNN classifier from Part 1. For each bean, it finds the nearest training examples and predicts the most common class among them. If there is a tie, the class of the closest neighbor wins. We measured performance using macro precision, recall, and F1, giving each of the seven classes equal weight. We first tested all 16 features, then used forward feature selection with GridSearchCV and 10-fold cross-validation, searching odd values of k from 1 to 41.

With seven possible labels, a prediction is not just a yes-or-no vote. We count the neighbors for each bean type and choose the label with the most votes. Macro F1 was helpful because the class sizes differ: it calculates an F1 score for each type and then gives all seven types equal weight, rather than letting common types dominate the summary.

Results

Using all 16 features, the best cross-validated macro F1 was 0.9370 at k = 9. Forward selection gave a slightly higher score of 0.9400 with ten features and k = 7. The chosen features were Perimeter, ShapeFactor2, roundness, Compactness, ShapeFactor4, ShapeFactor1, Solidity, EquivDiameter, ShapeFactor3, and MinorAxisLength. Removing six features slightly improved the score while making the model simpler.

The difference between 0.9370 and 0.9400 is only 0.0030 F1 points. That is a small improvement, so the more meaningful practical result is that the selected model used six fewer measurements and still performed at least as well in training cross-validation. This comparison does not by itself prove that the new feature set would win on every future dataset.

Model	Best k	CV macro F1
All 16 features	9	0.9370
Selected 10 features	7	0.9400

Table 1. Baseline compared with the selected feature set.

After choosing the features and k, we used 10-fold cross-validation on the training data to build ten models. Each was trained on nine folds and evaluated on the same separate test set of 2,723 beans. The scores for all ten models are shown below.

Fold	Precision	Recall	F1
1	0.9347	0.9334	0.9340
2	0.9348	0.9338	0.9343
3	0.9333	0.9310	0.9321
4	0.9343	0.9326	0.9334
5	0.9332	0.9316	0.9324
6	0.9351	0.9331	0.9341
7	0.9349	0.9338	0.9343
8	0.9380	0.9363	0.9371
9	0.9374	0.9359	0.9366
10	0.9347	0.9325	0.9335
Mean	**0.9351**	**0.9334**	**0.9342**

Table 2. Test-set macro precision, recall, and F1 scores for all ten models.

Discussion of results

kNN performed strongly on this dataset, with a mean test macro F1 of 0.9342. Feature selection made a much smaller difference here than it did for the heart disease dataset in Part 1. Even with only three features—Perimeter, ShapeFactor2, and roundness—the cross-validated score reached about 0.927, and the improvement became small after roughly seven features.

One possible reason is that several bean measurements describe similar aspects of shape and size. For example, Area and ConvexArea are closely related. The ten test scores were also very similar, partly because every model was evaluated on the same large test set and trained on largely overlapping data. These scores are therefore not ten independent test sets.

Comparing the two parts, feature selection helped much more for heart disease than for dry beans. For this dataset, we learned that it can still be useful even when the improvement in F1 is small, because we can use fewer features without giving up predictive performance. The two F1 scores describe different classification problems, though, so their difference alone does not prove that one task is inherently easier.

There are also limits to this comparison. Part I uses F1 for a binary disease label, while Part II uses macro F1 over seven bean classes, so the numbers do not measure identical tasks. The ten test-set rows also come from models tested on the same held-out beans, not ten independent new test samples. We therefore treat the results as evidence about these particular experiments rather than a general ranking of the two datasets.